

QUESTIONS PFE:

- What is the purpose of this platform?
 - A: To support conference-style peer review by automating ingestion, metadata extraction, semantic indexing, reviewer suggestion, plagiarism screening, and reviewer support.
- Is this for conference review or journal review?
 - A: It's designed for conference review: faster cycles, lighter review forms, and emphasis on short turn-around and program-committee style decisions.
- How does conference review differ from journal review?
 - A: Conferences prioritize speed and program fit; journals prioritize depth, revisions, and archival record — the platform optimizes for the conference workflow.
- What triggers the AI pipeline?
 - A: Paper submission triggers ingestion and async embedding; reviewer actions (summary, plagiarism, chat) trigger review-stage agents.
- What exactly is LangGraph in your system?
 - A: LangGraph is used conceptually to structure stateful, conditionally-branching agents inside the FastAPI AI service that orchestrate parsing, indexing, summarization, plagiarism, matching, and chat.
- How is LangGraph different from LangChain?
 - A: LangChain is a toolkit for building chains/RAG in Python; LangGraph emphasizes stateful agent graphs and conditional routing — we used a LangGraph-style pipeline for orchestrated multi-agent flows.
- Is RAG used for the chatbot?
 - A: Not in the strict sense; the chat is paper-grounded (uses title/abstract/keywords) and semantic search is available, but we do not perform full multi-document RAG retrieval for answers.
- Which embedding model do you use and why?

- A: We use [nomic-embed-text](#) via Ollama for local inference because it runs locally, is efficient, and suits semantic search; it avoids external API dependencies.
- What is the embedding dimensionality?
 - A: Dimensionality is model-dependent (commonly 1536 for [nomic-embed-text](#) variants); our system records whatever the model reports at embedding time.
- Why store vectors in PostgreSQL instead of a separate vector DB?
 - A: Using PostgreSQL+pgvector keeps data and vectors in one transactional store, simplifies deployment and backup, and avoids operating a separate service.
- How do you compute similarity / plagiarism?
 - A: We compute cosine similarity between embeddings; plagiarism is an internal, corpus-only check that flags high similarity (threshold ~0.4).
- Is the plagiarism check a legal/external plagiarism detector?
 - A: No — it is an internal duplicate/overlap detector within the platform, not a replacement for cross-publisher plagiarism services.
- Which LLM do you use for summarization and why?
 - A: We use a Mistral model hosted locally via Ollama because it provides good summarization quality for local inference and privacy without calling external APIs.
- Why Ollama?
 - A: Ollama enables local model hosting, low latency, and consistent behavior without exposing data to cloud APIs.
- Why Docker/containers?
 - A: Containers provide reproducible environments, isolation, and simpler deployment across machines and CI.
- Why Angular 19?
 - A: Modern features, strong Angular Material compatibility, and good ecosystem support for building a responsive SPA.

- Why Spring Boot 4 and Java 21?
 - A: Latest Spring Boot offers updated APIs and performance; Java 21 provides modern language features and long-term support compatibility.
- Why PostgreSQL 18?
 - A: Stable relational DB with pgvector support, transactionality and strong tooling for backups and security.
- Why Python + FastAPI for AI service?
 - A: FastAPI is lightweight, async-friendly, and well-suited to hosting model endpoints and orchestrating agent pipelines.
- What is GROBID used for?
 - A: Robust TEI/XML extraction of bibliographic metadata and structured paper content from PDFs.
- What is [AiService.embedPaperAsync](#)?
 - A: A background task in Spring that posts paper metadata to /embed on the AI service to create and store embeddings asynchronously.
- How often are embeddings updated?
 - A: On submission or metadata change — embeddings are created/updated asynchronously at ingestion time.
- How do you combine semantic similarity and expertise for reviewer matching?
 - A: The AI search returns semantic candidates (cosine similarity); backend applies expertise keyword overlap and workload filters to produce final ranked suggestions.
- What happens if the AI service is down?
 - A: The backend falls back to a manual expertise-based candidate list and continues core functionality.
- Why choose Scrum for development if you worked alone?
 - A: Scrum provided structure, incremental delivery, and traceability; the supervisor recommended it to demonstrate realistic team workflows and planning discipline.

- What are the main limitations of your system?
 - A: Limited external plagiarism coverage, chat constrained to single-paper context, model quality depends on local models, and embedding/model dims are model-dependent.
- How do you ensure reviewer privacy and data security?
 - A: Data stays on-premises; JWT secures API calls; local model hosting avoids sending data to external APIs.
- How do you evaluate the AI outputs?
 - A: Manual spot checks, reviewer feedback, and threshold-based flags for similarity; future work includes quantitative evaluation with labeled cases.
- Why not use a cloud-hosted vector DB or LLM APIs?
 - A: To preserve privacy, reduce external costs, and ensure reproducibility; local hosting gives control for research settings.
- What is the role of pgvector?
 - A: It provides vector storage and similarity operators inside PostgreSQL for efficient semantic search.
- Why two abstracts: GROBID vs AI summary?
 - A: GROBID extracts the authors' original abstract (factual metadata); the AI summary is a reviewer-focused concise interpretation (contributions, methods, weaknesses).
- Why the similarity threshold of ~0.4?
 - A: Empirical trade-off between false positives and negatives for corpus duplicate detection; threshold can be tuned with labeled examples.
- How scalable is this architecture?
 - A: Service-oriented design allows independent scaling of backend and AI service; PostgreSQL may need horizontal strategies or a dedicated vector store at scale.

Cheat-sheet — 10 must-know Q&A (short answers)

- **Project purpose:** A conference peer-review platform automating ingestion, metadata extraction, semantic indexing, reviewer suggestion, plagiarism checks, summaries and reviewer Q&A.
- **Conference vs journal:** Conferences need fast turnarounds and program-fit decisions; journals need deeper review and revisions — the platform targets conference workflows.
- **AI orchestration (LangGraph):** LangGraph structures stateful agent nodes (parse → index → summarize → match → check → chat) inside the FastAPI AI service.
- **LangChain / RAG:** We do not run full multi-document RAG for chat; LangChain is a toolkit for chains/RAG, LangGraph is used here for conditional agent graphs and orchestration.
- **Embedding model & hosting:** We use [nomic-embed-text](#) via Ollama for local embeddings — chosen for privacy, local control, and efficiency; dimensionality is model-dependent and recorded at runtime.
- **Vector storage:** We use PostgreSQL + pgvector to keep vectors and relational data together for transactional simplicity and easier operations/backups.
- **Reviewer matching:** Candidates come from semantic search (cosine similarity); backend applies expertise overlap and workload filters — final ranking = semantic + rule-based filter.
- **Plagiarism check:** Internal, corpus-only cosine similarity check (flag threshold ≈ 0.4) to detect duplicates within the platform — not a publisher/global plagiarism service.
- **GROBID vs AI summary:** GROBID extracts the authors' original structured metadata/abstract; the AI summary is reviewer-oriented (contribution, methods, weaknesses).

- **Deployment & security:** Services run in containers for reproducibility; JWT secures APIs; models hosted locally (Ollama) to keep data on-premises and preserve privacy.

Challenges

- **Multi-agent orchestration:** implementing a stateful LangGraph-style pipeline and coordinating conditional agent flows reliably.
- **PDF parsing quality:** noisy PDFs and TEI variability from GROBID make metadata extraction brittle.
- **Embedding/model variability:** differing model dimensions and tuning (similarity threshold) require runtime checks and calibration.
- **Local model hosting constraints:** Ollama/Mistral running locally needs CPU/GPU resources, increases deployment complexity and latency.
- **Async consistency:** asynchronous embedding/indexing introduces eventual-consistency issues and error/retry handling.
- **Evaluation gap:** no large labeled dataset for objectively measuring reviewer matching, plagiarism, or summary quality.
- **Scalability of vector storage:** PostgreSQL+pgvector is convenient but has scaling limits versus dedicated vector DBs for large corpora.
- **Privacy vs. operability trade-offs:** on-prem models protect data but complicate updates and third-party improvements.
- **Single-developer constraints:** limited time for testing, extensive integration work, and implementing Scrum alone.
- **Human-in-the-loop UX:** tuning automation so suggestions help reviewers without causing mistrust or extra workload.